

Froggy CrackMe (Libertas per Croack) — Writeup

Author: Chris Noble

Introduction

This crackme is a Linux ELF64 PIE executable written in C++. The challenge validates a username and serial combination using several layers of hashing and nonlinear mixing.

The binary includes:

- PIE + ASLR
- Anti-debugging checks
- Username-dependent serial validation
- FNV-1a hashing
- MurmurHash-style avalanche mixers
- 128-bit serial packing
- Final constant comparison

The goal was to derive a valid serial mathematically without patching the binary.

Initial Recon

I started by opening the binary in radare2:

```
r2 -A ./froggy_crackme
```

Running `iz` revealed several useful strings:

- `/proc/self/status`
- `TracerPid:`
- `Debugger detected. The swamp rejects you.`
- `Serial format: XXXXXXXX-XXXXXXX-XXXXXXX-XXXXXXX`
- success/failure messages

This immediately revealed:

1. The binary used anti-debugging.
 2. The serial consisted of four hexadecimal blocks.
-

Anti-Debugging

The binary opened `/proc/self/status` and searched for `TracerPid:`.

If `TracerPid` was nonzero, execution terminated with:

```
Debugger detected. The swamp rejects you.
```

Instead of patching the check, I continued mostly through static analysis.

Locating the Validation Logic

The success string:

```
[+] The Swamp Gate opens.
```

was cross-referenced using:

```
axt @ <string_address>
```

This led to the final comparison logic.

Searching for the target constant using little-endian bytes:

```
/x c9587559339922b9
```

revealed the final compare block.

The comparison was:

```
cmp rcx, rax
jne fail
```

where:

```
rax = 0xb9229933597558c9
```

Meaning the final computed value in `rcx` must equal:

```
0xb9229933597558c9
```

Serial Parsing

The serial parser used:

```
%8x-%8x-%8x-%8x
```

meaning the serial consisted of:

```
A-B-C-D
```

where each block was a 32-bit hexadecimal integer.

The binary then packed the values into two 64-bit integers:

```
r9  = (A << 32) | B  
r14 = (C << 32) | D
```

This effectively made the serial a 128-bit input.

FNV-1a Hashing

The next discovery came from the constants:

```
0x14650fb0739d0383  
0x100000001b3
```

These are the standard 64-bit FNV-1a constants.

The binary hashed the username byte-by-byte.

The approximate structure was:

```
h ^= byte;  
h *= 0x100000001b3;
```

The binary also created a reversed copy of the username and hashed the reversed string again.

This produced two independent FNV hash states.

Reverse Username Construction

The reverse-string creation used SSE instructions and optimized copying loops.

The important discovery was that the binary generated:

```
reverse(username)
```

and performed a second FNV-1a pass over the reversed string.

This meant the serial depended on:

- username hash
- reversed username hash

making the challenge a real keygenme instead of a static serial check.

Avalanche Mixers

The constants:

```
0x9e3779b185ebca87  
0xc2b2ae3d27d4eb4f
```

identified MurmurHash-style avalanche finalizers.

The mixer structure looked like:

```
x ^= x >> 33;  
x *= CONST1;
```

```
x ^= x >> 29;  
x *= CONST2;
```

The binary applied multiple avalanche stages to both hash paths.

Important Mixer Constants

The final mixer also used several custom constants:

```
0xa3b1957c4d2e1901  
0xc0d0e0f112233445  
0x1337f00dcafebab  
0x2ea07040acdb444c  
0x56e57f5f77891a00
```

These constants participated in XOR folding and nonlinear mixing.

The critical discovery from assembly was:

```
xor rbp, r8
```

where:

```
r8 = 0xa3b1957c4d2e1901
```

meaning the first FNV hash path was initialized as:

```
rbp = fnv(name) ^ 0xa3b1957c4d2e1901;
```

Similarly:

```
rbx = fnv(reverse(name)) ^ 0xc0d0e0f112233445;
```

This detail was critical for building a correct solver.

Final Validation Logic

After avalanche stages, the serial values were folded into the hash state:

```
r9 ^= rdx;  
r14 ^= rdi;
```

The binary then performed:

- XOR mixing
- shifting
- multiplication
- additions
- another avalanche stage

before finally comparing the result to:

```
0xb9229933597558c9
```

Initial Failed Attempt

Initially I attempted:

- partial brute force in C
- approximate reconstruction of the mixer

The generated serials were close but invalid.

The failure came from modeling the avalanche ordering incorrectly.

The exact assembly translation mattered.

Symbolic Solving with Z3

The correct solution was to model the validation logic using Z3 bitvectors.

Why Z3 worked well:

- all operations were SMT-friendly

- 64-bit modular arithmetic maps naturally to BitVecs
- XORs/shifts/multiplications are easy for Z3

The solver treated the serial blocks symbolically:

```
A = BitVec('A', 64)
B = BitVec('B', 64)
C = BitVec('C', 64)
D = BitVec('D', 64)
```

The username chosen for solving was:

```
A
```

because a short username simplified constraints.

The final solver constraint was:

```
final == 0xb9229933597558c9
```

Z3 successfully solved the equations and produced a valid serial.

Valid Credentials

Username:

```
A
```

Serial:

```
25C73DE8-10114BD1-39BC16A4-CDE261D9
```

Program output:

```
[+] The Swamp Gate opens.
    Welcome, A. Froggy approves.
```

Final Solver Script

```
from z3 import *

TARGET = BitVecVal(0xb9229933597558c9, 64)

FNV_OFFSET = BitVecVal(0x14650fb0739d0383, 64)
FNV_PRIME = BitVecVal(0x100000001b3, 64)

MUL1 = BitVecVal(0x9e3779b185ebca87, 64)
MUL2 = BitVecVal(0xc2b2ae3d27d4eb4f, 64)

CONST_RBP = BitVecVal(0xa3b1957c4d2e1901, 64)
CONST_RBX = BitVecVal(0xc0d0e0f112233445, 64)

CONST1 = BitVecVal(0x1337f00dcafebab, 64)
CONST2 = BitVecVal(0x2ea07040acdb444c, 64)
CONST3 = BitVecVal(0x56e57f5f77891a00, 64)

USERNAME = b"A"
REV_USERNAME = USERNAME[::-1]

def fnv1a(data):
    h = FNV_OFFSET

    for b in data:
        h = h ^ BitVecVal(b, 64)
        h = h * FNV_PRIME

    return h

A = BitVec("A", 64)
B = BitVec("B", 64)
C = BitVec("C", 64)
D = BitVec("D", 64)

s = Solver()

for v in [A, B, C, D]:
    s.add(v >= 0)
    s.add(v <= 0xffffffff)

r9 = (A << 32) | B
r14 = (C << 32) | D

h1 = fnv1a(USERNAME)
```



```

h2 = fnv1a(REV_USERNAME)

rbp = h1 ^ CONST_RBP
rbx = h2 ^ CONST_RBX

rcx1 = rbp ^ LShR(rbp, 33)
rdx1 = rcx1 * MUL1

tmp1 = rdx1 ^ LShR(rdx1, 29)
rdx2 = tmp1 * MUL2

rsi1 = rbx ^ LShR(rbx, 33)
rsi2 = rsi1 * MUL1

tmp2 = rsi2 ^ LShR(rsi2, 29)
rdi1 = tmp2 * MUL2

r9_mix = r9 ^ rdx2
rdx3 = LShR(rdx2, 32) ^ r9_mix

r14_mix = r14 ^ rdi1
rdi2 = LShR(rdi1, 32) ^ r14_mix

rsi3 = CONST1 ^ rdi2

shift_a = rsi3 << 17
shift_b = LShR(rsi3, 3)

r8 = shift_a ^ shift_b

rdi3 = CONST2 ^ rdx3
rdi3 = rdi3 ^ r8

r8a = rdi3 ^ LShR(rdi3, 33)
r8b = r8a * MUL1

r8c = r8b ^ LShR(r8b, 29)
r8d = r8c * MUL2

r8e = LShR(r8d, 32) ^ r8d

rdx4 = rdx3 << 7

rdi4 = CONST3 ^ rdx4

rsi4 = LShR(rsi3, 5)
rsi4 = rsi4 + rdi4
rsi4 = rsi4 + r8e

```

```

final1 = rsi4 ^ LShR(rsi4, 33)
final2 = final1 * MUL1

final3 = final2 ^ LShR(final2, 29)
final4 = final3 * MUL2

final  = LShR(final4, 32) ^ final4

s.add(final == TARGET)

print("[*] solving...")

if s.check() == sat:

    m = s.model()

    a = m[A].as_long()
    b = m[B].as_long()
    c = m[C].as_long()
    d = m[D].as_long()

    serial = f"{a:08X}-{b:08X}-{c:08X}-{d:08X}"

    print("\n[+] VALID SERIAL FOUND\n")

    print("Username :", USERNAME.decode())
    print("Serial   :", serial)

else:
    print("unsat")

```

Conclusion

This challenge was an excellent example of a modern Linux keygenme.

The binary combined:

- anti-debugging
- hashing
- reverse-string processing
- Murmur-style avalanche mixing
- nonlinear arithmetic
- symbolic constraints

The correct solution path was:

1. reverse the validator
2. recover exact mixer semantics
3. translate assembly into BitVec equations
4. solve symbolically with Z3

No patching was required.